Real-Time UPI Fraud Detection System

*Complete Technical Implementation Guide by **Unknsoul***

For Intermediate Python & ML Developers

100% Free & Open-Source Stack • Simulated Data • Fully Buildable

1. Introduction
2. System Overview
3. Tech Stack (Free Only)
4. Data Simulation
5. Feature Engineering
6. Machine Learning Models
7. Model Training
8. Backend API Development
9. Real-Time Simulation
10. Decision Engine
11. Alert System
12. Database Design
13. Architecture Diagram
14. Performance Considerations
15. Limitations & Future Work
16. Step-by-Step Implementation Plan
17. Conclusion

1. Introduction

1.1 What is UPI Fraud?

Unified Payments Interface (UPI) is India's real-time digital payment system, processing over 10 billion transactions per month. UPI fraud refers to unauthorized or deceptive transactions made using another person's UPI account, phone, or credentials. Common fraud types include:

- Phishing attacks — fake payment links sent via SMS or WhatsApp
- SIM swapping — attacker takes over victim's phone number to intercept OTPs
- Social engineering — fraudsters impersonate bank officials to extract PIN
- Screen sharing scams — victim unknowingly shares screen showing UPI PIN
- Mule account networks — money laundered through chains of compromised accounts
- Merchant fraud — fake QR codes placed over legitimate ones in shops

1.2 Why Fraud Detection Matters

UPI fraud has grown 300% in 3 years. Annual losses exceed Rs. 2,000 crore. With 300+ million active UPI users — many of them first-time digital banking users in rural India — the impact on individuals is severe. A single fraudulent transaction can wipe out a week's earnings for a daily wage worker with no recourse.

1.3 Problems with Rule-Based Systems

Traditional fraud detection uses static rules like "block if amount > Rs. 50,000 from a new device". These systems fail because:

- Fraudsters learn and adapt rules quickly; they simply stay below thresholds
- Rules generate too many false positives, blocking legitimate transactions
- Novel fraud patterns (new attack vectors) are never covered by existing rules
- Rules cannot detect subtle behavioral anomalies across millions of users
- Maintenance is expensive — rules require constant manual updates

1.4 Goal: ML-Based Real-Time Detection

This guide builds a machine learning system that scores every UPI transaction in real time (under 50ms), identifies fraud patterns that rule-based systems miss, and adapts to new fraud vectors via retraining. The entire system uses free and open-source tools and can be built and tested on a laptop without any paid APIs.

> *Goal: Build a working fraud detection pipeline using Python, XGBoost, FastAPI, PostgreSQL, and Redis — entirely free and open-source, with simulated transaction data.*

2. System Overview

2.1 High-Level Architecture

The system has five major layers. Each layer has a clear responsibility and communicates with the next via APIs or message queues:

Layer 1: Data Input	Simulated transaction generator OR real app sending JSON to the API
Layer 2: Feature Extraction	Compute 30+ features from raw transaction data using Redis + Pandas
Layer 3: ML Models	XGBoost scores the transaction; optional LSTM for sequence patterns
Layer 4: Decision Engine	Applies thresholds: Allow / Flag (step-up auth) / Block
Layer 5: Storage & Alerts	PostgreSQL stores result; console/email alert for fraud events

2.2 End-to-End Transaction Flow

Here is what happens from the moment a user initiates a UPI payment to the final decision:

1. User initiates payment in UPI app (or simulated via Python script)
2. Transaction JSON is sent to FastAPI /predict endpoint
3. Feature extraction layer pulls user history from Redis and computes features
4. XGBoost model loads features and returns a fraud probability score (0.0 to 1.0)
5. Decision engine maps score to action: Allow, Flag for OTP, or Block
6. Result is stored in PostgreSQL transactions table with fraud label
7. If fraud detected, alert is triggered (console log / email / SMS in production)
8. Response is returned to the caller with risk_score and decision

2.3 Real-Time vs Batch Processing

Aspect	Real-Time (This System)	Batch Processing
Trigger	Per transaction, immediate	Hourly/daily job
Latency	< 50ms	Minutes to hours
Use Case	Block fraud before it completes	Analytics, model retraining
Tech	FastAPI + Redis	Airflow + Pandas
Complexity	Higher	Lower

3. Tech Stack (Free & Open-Source Only)

Every tool in this stack is 100% free to use. No paid APIs, no cloud costs, no licenses required.

Tool	Version	Role	Why This Tool
Python	3.10+	Core language	Rich ML ecosystem, fast prototyping
FastAPI	0.110+	REST API server	Fastest Python web framework, auto-generates docs
XGBoost	2.0+	Primary fraud classifier	Best performance on tabular data, very fast inference
Scikit-learn	1.4+	Preprocessing, metrics	Standard ML utilities, SMOTE for class imbalance
Pandas	2.0+	Feature engineering	Powerful dataframe operations for computing features
PostgreSQL	15+	Persistent data store	Free, production-grade relational database
Redis	7+	Feature store / cache	Sub-millisecond key lookups for velocity features
Joblib	1.3+	Model serialization	Save/load trained models efficiently
Apache Kafka	3.6+	Streaming (optional)	Handles 100K+ events/sec for advanced setup
Jupyter	6+	Exploration / training	Interactive development for ML experiments

Installation — All at Once

```
# Create a virtual environment
python -m venv fraud_env
source fraud_env/bin/activate   # Windows: fraud_env\Scripts\activate

# Install all Python dependencies
pip install fastapi uvicorn xgboost scikit-learn pandas
pip install redis psycopg2-binary joblib imbalanced-learn
pip install faker numpy python-dotenv

# Start PostgreSQL (Docker)
docker run -d --name fraud_pg -e POSTGRES_PASSWORD=secret -p 5432:5432 postgres:15

# Start Redis (Docker)
docker run -d --name fraud_redis -p 6379:6379 redis:7
```

4. Data Simulation

4.1 Why We Simulate Data

Real UPI transaction data from NPCI is restricted — it contains PII (personal identifiable information) and requires bank partnerships and regulatory agreements to access. For building and testing this system, we generate synthetic data that mirrors the structure and statistical distribution of real transactions.

> Important: Never use real user data for testing. Always use synthetic or anonymized datasets in development environments.

4.2 Transaction JSON Structure

Each simulated transaction has the following fields:

```
{
  "txn_id":       "TXN_20240115_abc123",   // Unique transaction ID
  "sender_vpa":   "user123@okaxis",         // Sender UPI Virtual Payment Address
  "receiver_vpa": "merchant@paytm",         // Receiver VPA
  "amount":       2500.00,                  // Transaction amount in INR
  "timestamp":    "2024-01-15T14:32:11Z",   // ISO 8601 UTC timestamp
  "device_id":    "DEV_iPhone14_xyz",       // Hashed device identifier
  "ip_address":   "103.21.58.44",           // Sender IP address
  "location":     {"lat": 19.07, "lon": 72.87},  // GPS coordinates
  "merchant_cat": "grocery",                // Merchant category code
  "channel":      "mobile_app"             // mobile_app / web / IVR
}
```

4.3 Python Data Generator

```
# simulate_transactions.py
import random, uuid, json
from faker import Faker
from datetime import datetime

fake = Faker('en_IN')

BANKS    = ['okaxis', 'oksbi', 'okicici', 'paytm', 'ybl']
CATS     = ['grocery', 'food', 'travel', 'recharge', 'utility', 'shopping']
CHANNELS = ['mobile_app', 'web', 'IVR']

def generate_transaction(is_fraud=False):
    amount = random.uniform(50000, 200000) if is_fraud else random.uniform(10, 25000)
    return {
        "txn_id":       f"TXN_{uuid.uuid4().hex[:12]}",
```

```
        "sender_vpa":   f"{fake.user_name()}@{random.choice(BANKS)}",
        "receiver_vpa": f"{fake.user_name()}@{random.choice(BANKS)}",
        "amount":       round(amount, 2),
        "timestamp":    datetime.utcnow().isoformat() + "Z",
        "device_id":    f"DEV_{uuid.uuid4().hex[:8]}",
        "ip_address":   fake.ipv4_public(),
        "location":     {"lat": fake.latitude(), "lon": fake.longitude()},
        "merchant_cat": random.choice(CATS),
        "channel":      random.choice(CHANNELS),
        "is_fraud":     1 if is_fraud else 0
    }

# Generate 100,000 transactions (1% fraud rate)
dataset = ([generate_transaction(False) for _ in range(99000)] +
           [generate_transaction(True)  for _ in range(1000)])
random.shuffle(dataset)

with open('transactions.json', 'w') as f:
    json.dump(dataset, f)
print(f'Generated {len(dataset)} transactions')
```

4.4 Kaggle Datasets for Training

For more realistic training data, use these free Kaggle datasets:

- Credit Card Fraud Detection (Kaggle) — 284K transactions, 492 fraud cases, well-labeled
- IEEE-CIS Fraud Detection — 590K transactions, rich feature set, closest to UPI patterns
- Synthetic Financial Datasets (PaySim) — 6.3M mobile money transactions, 8,213 fraud

> *Download: kaggle.com/datasets — search 'credit card fraud detection'. These datasets have similar structure to UPI transactions and can be adapted with column renaming.*

5. Feature Engineering

Features are the inputs your model actually learns from. Raw transaction data is not enough — you need to compute derived signals that capture fraud patterns. Feature quality is more important than model complexity.

5.1 Transaction Features

- amount — raw INR value. Fraud transactions often have unusually high or low amounts
- hour_of_day — derived from timestamp. Fraud spikes at 2-5 AM
- day_of_week — weekday vs weekend. Different fraud patterns apply
- is_new_receiver — 1 if sender has never paid this receiver before
- merchant_cat_risk — encoded risk score per merchant category (1=low, 5=high)

5.2 Behavioral / Velocity Features

These are the most powerful fraud signals. They compare the current transaction against the user's own history:

- txn_count_1min — how many transactions in the last 60 seconds from this sender
- txn_count_1hr — transactions in the last hour. Fraud rings move money fast
- total_amount_24hr — total amount sent in last 24 hours by this sender
- avg_amount_30d — 30-day rolling average. Flag if current amount > 5x average
- unique_receivers_7d — number of unique receivers in last 7 days. High = suspicious

5.3 Device & Location Features

- is_new_device — 1 if device_id not seen in last 30 days
- is_vpn_ip — 1 if IP is from known VPN/proxy ASN range
- distance_from_usual_location — km from user's most common GPS region
- impossible_travel — 1 if user transacted from location > 900km away in < 1 hour

5.4 Computing Features with Pandas + Redis

```
# feature_engineering.py
import pandas as pd
import redis
from datetime import datetime, timedelta

r = redis.Redis(host='localhost', port=6379, decode_responses=True)
```

```
MERCHANT_RISK = {'grocery':1, 'food':1, 'utility':2, 'shopping':3,
                 'travel':3, 'recharge':2, 'crypto':5, 'unknown':4}

def extract_features(txn: dict) -> dict:
    vpa   = txn['sender_vpa']
    amt   = txn['amount']
    ts    = datetime.fromisoformat(txn['timestamp'].replace('Z',''))

    # Time features
    hour  = ts.hour
    dow   = ts.weekday()

    # Velocity from Redis
    key_1m  = f'vel:{vpa}:1m'
    key_1h  = f'vel:{vpa}:1h'
    key_24h = f'amt:{vpa}:24h'
    cnt_1m  = int(r.get(key_1m)  or 0)
    cnt_1h  = int(r.get(key_1h)  or 0)
    amt_24h = float(r.get(key_24h) or 0)

    # Average amount from Redis (stored as running avg)
    avg_30d = float(r.get(f'avg:{vpa}:30d') or amt)
    ratio   = amt / avg_30d if avg_30d > 0 else 1.0

    features = {
        'amount':           amt,
        'hour_of_day':      hour,
        'day_of_week':      dow,
        'merchant_cat_risk': MERCHANT_RISK.get(txn['merchant_cat'], 3),
        'txn_count_1min':   cnt_1m,
        'txn_count_1hr':    cnt_1h,
        'total_amount_24hr': amt_24h,
        'amount_vs_avg_ratio': ratio,
        'is_new_device':    int(r.get(f'dev:{vpa}:{txn["device_id"]}') is None),
        'is_night_txn':     int(hour < 5 or hour > 23),
    }
    return features

def update_redis_counters(txn: dict):
    vpa = txn['sender_vpa']
    amt = txn['amount']
    pipe = r.pipeline()
    # Increment velocity counters with TTL
    pipe.incr(f'vel:{vpa}:1m');  pipe.expire(f'vel:{vpa}:1m', 60)
    pipe.incr(f'vel:{vpa}:1h');  pipe.expire(f'vel:{vpa}:1h', 3600)
    pipe.incrbyfloat(f'amt:{vpa}:24h', amt)
    pipe.expire(f'amt:{vpa}:24h', 86400)
    # Mark device as seen (7-day TTL)
    pipe.set(f'dev:{vpa}:{txn["device_id"]}', 1, ex=604800)
    pipe.execute()
```

6. Machine Learning Models

6.1 XGBoost — Primary Model

XGBoost (Extreme Gradient Boosting) is the best model for fraud detection on tabular data. It builds an ensemble of decision trees sequentially, where each tree corrects the errors of the previous one. It handles missing values natively, is fast (< 5ms inference), and produces probability scores.

Algorithm	Gradient Boosted Decision Trees (GBDT)
Output	Probability score between 0.0 (legitimate) and 1.0 (fraud)
Strengths	Handles imbalanced data, fast, interpretable, top Kaggle performer
Weaknesses	Does not capture sequential user behavior over time
Key Hyperparameters	n_estimators (200-500), max_depth (4-8), learning_rate (0.05-0.1)

6.2 Handling Imbalanced Data

In real UPI data, fraud is only ~0.1% of all transactions. If you train naively, the model predicts everything as legitimate (99.9% accuracy, 0% fraud recall — useless). Solutions:

- SMOTE (Synthetic Minority Oversampling) — generates synthetic fraud samples to balance classes
- scale_pos_weight parameter in XGBoost — weights fraud class higher during training
- Threshold tuning — adjust decision threshold below 0.5 to increase fraud recall

6.3 Optional: LSTM for Behavioral Sequences

An LSTM (Long Short-Term Memory) model captures how a user's spending changes over time. It takes the last N transactions as a sequence and flags sudden behavioral changes.

- Input: sequence of last 20 transactions per user [amount, category, hour, day_of_week]
- Output: anomaly score 0.0 to 1.0 for the current transaction
- When to use: when you have 90+ days of user history; adds 5-10% AUC improvement

> For beginners: Start with XGBoost only. Add LSTM in Phase 2 once the core system is working.

6.4 Model Evaluation Metrics

Metric	Formula	Target	What It Means
Accuracy	TP+TN / Total	> 95%	Overall correctness —

			misleading with imbalanced data
Precision	TP / (TP+FP	> 90%	Of all flagged transactions, how many are actually fraud
Recall	TP / (TP+FN)	> 80%	Of all actual fraud, how many did we catch
F1-Score	2 * P*R/(P+R)	> 85%	Harmonic mean of precision and recall
AUC-ROC	Area under ROC curve	> 0.97	Model's ability to distinguish fraud from legitimate

7. Model Training

7.1 Full Training Pipeline

```
# train_model.py
import pandas as pd
import numpy as np
import joblib
from xgboost import XGBClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, roc_auc_score
from sklearn.preprocessing import LabelEncoder
from imblearn.over_sampling import SMOTE

# ── 1. Load data ──────────────────────────────────────
df = pd.read_json('transactions.json')

# ── 2. Feature engineering ────────────────────────────
df['hour_of_day']       = pd.to_datetime(df['timestamp']).dt.hour
df['day_of_week']       = pd.to_datetime(df['timestamp']).dt.dayofweek
df['is_night_txn']      = ((df['hour_of_day'] < 5) | (df['hour_of_day'] >
23)).astype(int)

MERCHANT_RISK = {'grocery':1,'food':1,'utility':2,'shopping':3,'travel':3,
                 'recharge':2,'crypto':5,'unknown':4}
df['merchant_cat_risk'] = df['merchant_cat'].map(MERCHANT_RISK).fillna(3)

# Simulate velocity features (in production, these come from Redis)
df['txn_count_1min']    = np.random.poisson(1.2, len(df))
df['txn_count_1hr']     = np.random.poisson(4.5, len(df))
df['total_amount_24hr'] = np.random.exponential(8000, len(df))
df['is_new_device']     = np.random.binomial(1, 0.08, len(df))
df['amount_vs_avg']     = df['amount'] / df.groupby('sender_vpa')
['amount'].transform('mean')

FEATURES = ['amount','hour_of_day','day_of_week','merchant_cat_risk',
            'txn_count_1min','txn_count_1hr','total_amount_24hr',
            'is_new_device','is_night_txn','amount_vs_avg']

X = df[FEATURES]
y = df['is_fraud']

# ── 3. Train / test split ─────────────────────────────
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# ── 4. Handle class imbalance with SMOTE ──────────────
smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X_train, y_train)
print(f'After SMOTE: {y_resampled.value_counts().to_dict()}')

# ── 5. Train XGBoost ──────────────────────────────────
model = XGBClassifier(
    n_estimators=300,
```

```
    max_depth=6,
    learning_rate=0.07,
    subsample=0.8,
    colsample_bytree=0.8,
    scale_pos_weight=10,   # Extra weight on fraud class
    use_label_encoder=False,
    eval_metric='auc',
    random_state=42
)
model.fit(X_resampled, y_resampled,
          eval_set=[(X_test, y_test)], verbose=50)

# ── 6. Evaluate ──────────────────────────────────────
y_pred  = model.predict(X_test)
y_proba = model.predict_proba(X_test)[:, 1]
print(classification_report(y_test, y_pred))
print(f'AUC-ROC: {roc_auc_score(y_test, y_proba):.4f}')

# ── 7. Save model and feature list ────────────────────
joblib.dump(model,    'models/fraud_model.pkl')
joblib.dump(FEATURES, 'models/feature_list.pkl')
print('Model saved to models/')
```

7.2 Hyperparameter Tuning with Optuna (Free)

```
# pip install optuna
import optuna
from sklearn.model_selection import cross_val_score

def objective(trial):
    params = {
        'n_estimators':  trial.suggest_int('n_estimators', 100, 500),
        'max_depth':     trial.suggest_int('max_depth', 3, 10),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.3, log=True),
        'subsample':     trial.suggest_float('subsample', 0.6, 1.0),
    }
    clf = XGBClassifier(**params, use_label_encoder=False, eval_metric='auc')
    scores = cross_val_score(clf, X_resampled, y_resampled, cv=3, scoring='roc_auc')
    return scores.mean()

study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=50)
print('Best params:', study.best_params)
```

8. Backend API Development

8.1 FastAPI Server with /predict Endpoint

```
# api/main.py
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import joblib, redis, numpy as np
from feature_engineering import extract_features, update_redis_counters
import psycopg2, os

app   = FastAPI(title='UPI Fraud Detection API', version='1.0')
model = joblib.load('models/fraud_model.pkl')
feats = joblib.load('models/feature_list.pkl')
r     = redis.Redis(host='localhost', port=6379, decode_responses=True)

# PostgreSQL connection
conn = psycopg2.connect(host='localhost', dbname='fraud_db',
                        user='postgres', password='secret')

class Transaction(BaseModel):
    txn_id:       str
    sender_vpa:   str
    receiver_vpa: str
    amount:       float
    timestamp:    str
    device_id:    str
    ip_address:   str
    merchant_cat: str
    channel:      str

@app.post('/predict')
async def predict(txn: Transaction):
    txn_dict = txn.dict()

    # Extract features
    features = extract_features(txn_dict)
    X = np.array([[features[f] for f in feats]])

    # Score with model
    risk_score = float(model.predict_proba(X)[0][1])

    # Decision engine
    if risk_score > 0.70:
        decision = 'BLOCK'
    elif risk_score > 0.35:
        decision = 'FLAG'
    else:
        decision = 'ALLOW'

    # Update Redis counters for this sender
    update_redis_counters(txn_dict)

    # Store in PostgreSQL
    with conn.cursor() as cur:
```

```
        cur.execute(
            """INSERT INTO transactions (txn_id,sender_vpa,amount,risk_score,decision)
               VALUES (%s,%s,%s,%s,%s)""",
            (txn.txn_id, txn.sender_vpa, txn.amount, risk_score, decision)
        )
        conn.commit()

    return {
        'txn_id':     txn.txn_id,
        'risk_score': round(risk_score, 4),
        'decision':   decision,
        'features':   features
    }

@app.get('/health')
async def health(): return {'status': 'ok', 'model': 'XGBoost v1.0'}
```

8.2 Run the API

```
uvicorn api.main:app --host 0.0.0.0 --port 8000 --reload

# API docs auto-generated at: http://localhost:8000/docs
```

8.3 Example API Request & Response

```
# Request (curl):
curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{
    "txn_id": "TXN_abc123",
    "sender_vpa": "user@okaxis",
    "receiver_vpa": "shop@paytm",
    "amount": 75000.00,
    "timestamp": "2024-01-15T02:15:00Z",
    "device_id": "DEV_newphone99",
    "ip_address": "185.220.101.1",
    "merchant_cat": "unknown",
    "channel": "mobile_app"
  }'

# Response:
{
  "txn_id":     "TXN_abc123",
  "risk_score": 0.8723,
  "decision":   "BLOCK",
  "features": {
    "amount": 75000.0, "hour_of_day": 2, "is_night_txn": 1,
    "merchant_cat_risk": 4, "is_new_device": 1, ...
  }
}
```

9. Real-Time Simulation

9.1 Transaction Generator Loop

```
# simulate_live.py
import requests, time, random
from simulate_transactions import generate_transaction

API_URL = 'http://localhost:8000/predict'

print('Starting live transaction simulation...')
print('-' * 60)

for i in range(500):   # Simulate 500 live transactions
    # 2% fraud rate in simulation
    is_fraud = random.random() < 0.02
    txn      = generate_transaction(is_fraud=is_fraud)

    response = requests.post(API_URL, json=txn)
    result   = response.json()

    label = '[FRAUD]' if is_fraud else '[LEGIT]'
    print(f"{label} TXN {txn['txn_id'][:12]} | "
          f"Amount: Rs.{txn['amount']:>10,.0f} | "
          f"Score: {result['risk_score']:.3f} | "
          f"Decision: {result['decision']}")

    time.sleep(0.1)   # 100ms between transactions (10 TPS)
```

9.2 Optional: Kafka Streaming Architecture

For production-grade streaming, replace the HTTP loop with Apache Kafka:

- Producer: Python script publishes transaction JSON to Kafka topic upi-transactions
- Consumer: Faust stream processor subscribes, enriches with Redis features, calls ML model
- Benefit: handles 100,000+ transactions per second, fault-tolerant, replays failed events

```
# kafka_producer.py (requires: pip install kafka-python)
from kafka import KafkaProducer
import json

producer = KafkaProducer(
    bootstrap_servers='localhost:9092',
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)

txn = generate_transaction(is_fraud=False)
producer.send('upi-transactions', value=txn)
producer.flush()
```

> Kafka requires more setup (Docker + Zookeeper). Start with the HTTP simulation loop first. Kafka is only needed when you need to handle 10,000+ transactions per second.

10. Decision Engine

10.1 Risk Score Thresholds

Risk Score Range	Decision	Action	User Experience
0.00 – 0.34	ALLOW	Transaction completes	Instant, no friction
0.35 – 0.69	FLAG	Trigger step-up auth	User must enter OTP to confirm
0.70 – 1.00	BLOCK	Transaction rejected	User informed, bank notified

10.2 Step-Up Authentication Simulation

```
# decision_engine.py

def make_decision(risk_score: float, txn: dict) -> dict:
    if risk_score < 0.35:
        return {'action': 'ALLOW', 'reason': 'Low risk transaction'}

    elif risk_score < 0.70:
        # Simulate OTP step-up (in real system, send OTP via SMS)
        otp_sent   = simulate_otp_send(txn['sender_vpa'])
        otp_correct = simulate_otp_verify()   # User input simulation
        if otp_correct:
            return {'action': 'ALLOW', 'reason': 'Step-up auth passed'}
        else:
            return {'action': 'BLOCK', 'reason': 'Step-up auth failed'}

    else:
        return {
            'action': 'BLOCK',
            'reason': 'High fraud probability',
            'top_signals': get_top_risk_signals(txn)
        }

def simulate_otp_send(vpa: str):
    otp = str(random.randint(100000, 999999))
    print(f'[OTP SIMULATION] Sending OTP {otp} to {vpa}')
    return otp

def simulate_otp_verify():
    return random.random() > 0.1  # 90% pass rate in simulation
```

10.3 Reducing False Positives

- Trusted merchant whitelist: never block payments to known large merchants (stored in Redis set)
- User history baseline: if amount_vs_avg < 3.0 and is_known_receiver, lower effective score by 0.15
- Progressive trust: accounts older than 2 years with clean history get a 0.10 score reduction
- Threshold tuning: regularly analyze FPR on real data; adjust thresholds per merchant category

11. Alert System

11.1 Console Logging (Development)

```
# alerts.py
import logging, smtplib
from email.mime.text import MIMEText
from datetime import datetime

logging.basicConfig(
    filename='fraud_alerts.log',
    level=logging.WARNING,
    format='%(asctime)s [%(levelname)s] %(message)s'
)

def send_alert(txn: dict, risk_score: float, decision: str):
    msg = (f"FRAUD ALERT | TXN: {txn['txn_id']} | "
           f"Sender: {txn['sender_vpa']} | Amount: Rs.{txn['amount']:,.0f} | "
           f"Score: {risk_score:.3f} | Decision: {decision}")

    # Console
    print(f'\u26a0\ufe0f  {msg}')

    # Log file
    logging.warning(msg)

    # Email (optional — use Gmail SMTP for testing)
    if decision == 'BLOCK':
        send_email_alert(msg)

def send_email_alert(body: str):
    # Configure SMTP in .env: EMAIL_USER, EMAIL_PASS
    try:
        msg = MIMEText(body)
        msg['Subject'] = 'UPI Fraud Alert'
        msg['From'] = 'alerts@yourbank.com'
        msg['To']   = 'security@yourbank.com'
        with smtplib.SMTP_SSL('smtp.gmail.com', 465) as server:
            server.login(EMAIL_USER, EMAIL_PASS)
            server.send_message(msg)
    except Exception as e:
        print(f'Email alert failed: {e}')
```

11.2 Real-World Alert Channels

- SMS: Twilio API or MSG91 — send instant fraud alerts to user's registered mobile
- Push notifications: Firebase FCM — in-app alerts with action buttons (Approve/Deny)
- Bank officer dashboard: Grafana panel shows real-time fraud queue requiring manual review
- Webhook to bank core system: POST request to bank's internal fraud management system

12. Database Design

12.1 PostgreSQL Schema

```
-- Create database
CREATE DATABASE fraud_db;
\c fraud_db;

-- Main transactions table
CREATE TABLE transactions (
    id            BIGSERIAL PRIMARY KEY,
    txn_id        VARCHAR(64)  UNIQUE NOT NULL,
    sender_vpa    VARCHAR(128) NOT NULL,
    receiver_vpa  VARCHAR(128) NOT NULL,
    amount        DECIMAL(12,2) NOT NULL,
    merchant_cat  VARCHAR(64),
    channel       VARCHAR(32),
    device_id     VARCHAR(128),
    ip_address    INET,
    timestamp     TIMESTAMPTZ  NOT NULL,
    risk_score    DECIMAL(5,4),
    decision      VARCHAR(10)  CHECK (decision IN ('ALLOW','FLAG','BLOCK')),
    is_confirmed_fraud BOOLEAN DEFAULT FALSE,
    created_at    TIMESTAMPTZ  DEFAULT NOW()
);

-- Fraud labels table (for retraining feedback)
CREATE TABLE fraud_labels (
    id         BIGSERIAL PRIMARY KEY,
    txn_id     VARCHAR(64) REFERENCES transactions(txn_id),
    label      BOOLEAN NOT NULL,   -- TRUE = fraud confirmed
    source     VARCHAR(32),        -- 'bank_officer', 'user_complaint', 'auto'
    labeled_at TIMESTAMPTZ DEFAULT NOW()
);

-- Model predictions log (for drift monitoring)
CREATE TABLE model_predictions (
    id           BIGSERIAL PRIMARY KEY,
    txn_id       VARCHAR(64),
    model_version VARCHAR(32),
    risk_score   DECIMAL(5,4),
    features_json JSONB,
    predicted_at  TIMESTAMPTZ DEFAULT NOW()
);

-- Indexes for fast lookups
CREATE INDEX idx_txn_sender ON transactions(sender_vpa, timestamp DESC);
CREATE INDEX idx_txn_decision ON transactions(decision, created_at DESC);
CREATE INDEX idx_txn_fraud ON transactions(is_confirmed_fraud) WHERE is_confirmed_fraud =
TRUE;
```

13. System Architecture Diagram

```
                    UPI FRAUD DETECTION SYSTEM — ARCHITECTURE
═══════════════════════════════════════════════════════════════════════

┌──────────────────────────────────────────────────────────────────┐
│                        DATA INPUT LAYER                          │
│   Python Simulator ──┬── WhatsApp Bot                            │
│                      ├── Mobile App (UPI)                        │
│                      └── Web Portal                              │
└──────────────────────────────────────────────────────────────────┘
                          │  HTTP POST /predict (JSON)
                          ▼
┌──────────────────────────────────────────────────────────────────┐
│                       FastAPI BACKEND                            │
│     /predict endpoint → validates → calls feature layer          │
└──────────────────────────────────────────────────────────────────┘
                  │                          │
                  ▼                          ▼
┌──────────────────────────┐   ┌──────────────────────────────────┐
│   REDIS Feature Store    │   │  FEATURE ENGINEERING MODULE      │
│  • Velocity counters     ◄───  Computes 30+ features in <5ms     │
│  • Rolling averages      │   │  (amount_vs_avg, velocity, etc)  │
│  • Device seen flags     │   └──────────────────────────────────┘
└──────────────────────────┘                   │
                                               ▼
┌──────────────────────────────────────────────────────────────────┐
│                        ML SCORING LAYER                          │
│   XGBoost Model ──▶ risk_score: 0.0-1.0                          │
│   (Optional) LSTM  ──▶ behavioral anomaly score                  │
│   Ensemble: 0.7*xgb + 0.3*lstm ──▶ final_score                   │
└──────────────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────────────┐
│                       DECISION ENGINE                            │
│   score < 0.35 ──▶ ALLOW   (transaction completes)               │
│   score 0.35-0.70 ──▶ FLAG (request OTP step-up)                 │
│   score > 0.70 ──▶ BLOCK   (reject + alert)                      │
└──────────────────────────────────────────────────────────────────┘
                  │                  │                  │
                  ▼                  ▼                  ▼
┌──────────────────┐   ┌──────────────────┐   ┌──────────────────┐
│ PostgreSQL DB    │   │ ALERT SYSTEM     │   │ API RESPONSE     │
│ Store result +   │   │ Console log      │   │ {risk_score,     │
│ fraud labels     │   │ Email / SMS      │   │  decision, features}
└──────────────────┘   └──────────────────┘   └──────────────────┘

═══════════════════════════════════════════════════════════════════════
RETRAINING LOOP (Weekly)
PostgreSQL confirmed fraud labels → Airflow DAG → Retrain XGBoost
→ Evaluate on holdout → Deploy if AUC improved → MLflow tracking
═══════════════════════════════════════════════════════════════════════
```

14. Performance Considerations

14.1 Latency Budget

Component	Target Latency	Optimization
Redis feature lookup	< 2ms	Use pipelining; keep counters in single hash
XGBoost inference	< 10ms	Load model once at startup; use numpy arrays
FastAPI request handling	< 5ms	Async endpoints; Pydantic validation
PostgreSQL write	< 15ms	Async DB write; use INSERT with RETURNING
Total end-to-end	< 50ms	All components in same datacenter/VPC

14.2 Model Optimization

- Load model once at API startup using joblib — never reload per request
- Use predict_proba with numpy arrays (not DataFrames) for fastest inference
- Enable XGBoost tree method='hist' for fastest CPU inference
- Set n_jobs=-1 during training; set nthread=2 during inference to avoid overload

14.3 Redis Optimization

- Use Redis pipelining to batch multiple counter reads/writes in one network call
- Store velocity counters as Redis HASHes per sender VPA — single key lookup
- Set appropriate TTLs: 1min=60s, 1hr=3600s, 24hr=86400s, 7day=604800s
- Use Redis Cluster for horizontal scaling beyond single-instance limits

14.4 Database Optimization

- Use async PostgreSQL with asyncpg library for non-blocking writes
- Partition the transactions table by month for query performance
- Archive data older than 90 days to cold storage (S3 / MinIO)
- Use read replicas for analytics queries — never hit the primary for reporting

15. Limitations & Future Improvements

15.1 Current Limitations

Limitation	Impact	Mitigation
No real NPCI API access	Cannot test with live UPI transactions	Use NPCI sandbox (requires bank partnership)
Simulated data gaps	Model may not capture all real fraud patterns	Supplement with Kaggle datasets
No behavioral biometrics	Missing device sensor signals	Add mobile SDK in next version
Single model (XGBoost)	Misses temporal behavior patterns	Add LSTM for sequence modeling
No graph analysis	Cannot detect fraud rings	Add PyG-based GNN module
No RBI compliance	Cannot deploy to production	Requires CERT-In audit + RBI approval

15.2 Future Improvements

- Graph Neural Networks (PyTorch Geometric) — detect mule account rings by analyzing the full UPI payment graph
- Behavioral Biometrics SDK — collect typing rhythm, touch pressure, gyroscope data from mobile app
- Online Learning — update model in real time as new fraud patterns emerge, without full weekly retraining
- Federated Learning — train across multiple banks without sharing raw transaction data
- SHAP Explainability UI — show customers and bank officers which features triggered the fraud flag
- Cloud deployment — containerize with Docker, deploy on AWS/GCP with auto-scaling
- Real NPCI API integration — requires formal partnership, PCI-DSS compliance, and CERT-In audit

16. Step-by-Step Implementation Plan

Phase	Week	Tasks	Deliverable
Phase 1: Data & ML	Week 1	Set up Python venv, install all packages, generate 100K synthetic transactions	transactions.json ready
Phase 1: Data & ML	Week 2	Feature engineering pipeline (Pandas + Redis counters simulation)	features_df.csv ready
Phase 1: Data & ML	Week 3	Train XGBoost with SMOTE, evaluate metrics, tune with Optuna	fraud_model.pkl saved
Phase 2: API	Week 4	Build FastAPI /predict endpoint, load model, Pydantic schemas	API running on port 8000
Phase 2: API	Week 5	Connect Redis for live velocity features, PostgreSQL writes, health endpoint	Full API stack working
Phase 3: Simulation	Week 6	Build live transaction simulator, decision engine, console alert system	End-to-end demo working
Phase 3: Simulation	Week 7	Add step-up OTP simulation, trusted merchant whitelist, FP reduction logic	Decision engine complete
Phase 4: Optimization	Week 8	Profiling, Redis pipelining, async PostgreSQL, model loading optimization	< 50ms latency achieved
Phase 4: Optimization	Week 9	Grafana dashboard, fraud rate monitoring, model drift detection	Monitoring dashboard live
Phase 4: Optimization	Week 10	Code cleanup, documentation, Dockerfile, README	Production-ready repo

> *Minimum Viable Product (MVP) is achievable in 3-4 weeks: data generation + XGBoost model + FastAPI /predict + console alerts. Phases 3-4 make it production-grade.*

17. Conclusion

17.1 What You Built

You have designed and can now implement a complete real-time UPI fraud detection system using 100% free, open-source tools. The system covers the full production pipeline:

- Data simulation that mirrors real UPI transaction structure
- Feature engineering with 30+ signals including velocity, behavioral, device, and location features
- XGBoost model trained with SMOTE to handle extreme class imbalance
- FastAPI scoring service with sub-50ms inference latency
- Redis-backed velocity counters for real-time feature computation
- Decision engine with three-tier thresholds and OTP step-up authentication
- PostgreSQL storage for transactions, fraud labels, and model audit trail
- Weekly retraining loop for continuous model improvement

17.2 Key Learnings

- Feature engineering matters more than model choice — good features beat complex models
- Class imbalance is the #1 training mistake — always use SMOTE or scale_pos_weight
- Redis is essential for real-time features — database queries are too slow at transaction speed
- False positive rate is as important as fraud detection — a system that blocks too many legitimate transactions is unusable
- Explainability is not optional — regulators and users both need to know why a transaction was blocked

17.3 Real-World Relevance

While this system uses simulated data, the architecture, models, features, and engineering patterns are identical to what major fintech companies and banks deploy in production. NPCI's own fraud detection systems use similar gradient boosted models, streaming pipelines, and graph analysis. Building this system gives you the skills to work on production fraud detection at scale.

The next step is to apply for NPCI Developer Sandbox access at developer.npci.org.in, which allows testing with controlled UPI transaction data under regulatory supervision. From there, a bank partnership and CERT-In security audit are the gateways to real production deployment.

> *Start with Week 1: generate transactions.json, install XGBoost, train your first model. The entire Phase 1 (data + trained model) can be completed in a single weekend.*